Matière : Framework Web	Discipline : 2ème GLID	Semestre : 2
Enseignant : Imene Lahyani	Année universitaire : 2024-2025	

Revision Framework Web

QCM: Cochez-la ou les bonnes réponses

Questions	Réponse(s)
1 La version Angular JS est la version	☐ V2 de Angular ☑ V1 de Angular
2. Dans le conteneur d'instances Angular, une nouvelle instance du service est crée à chaque fois qu'on l'injecte dans un composant	☐ vrai ☑ faux
3. Pour utiliser les courbes chartJs , il faut importer	☐ Ng2-charts ☑ ChartJS ☐ Ng-charts
4. A chaque chargement de la page, Angular charge :	☑ Le module Racine ☐ Le composant Racine

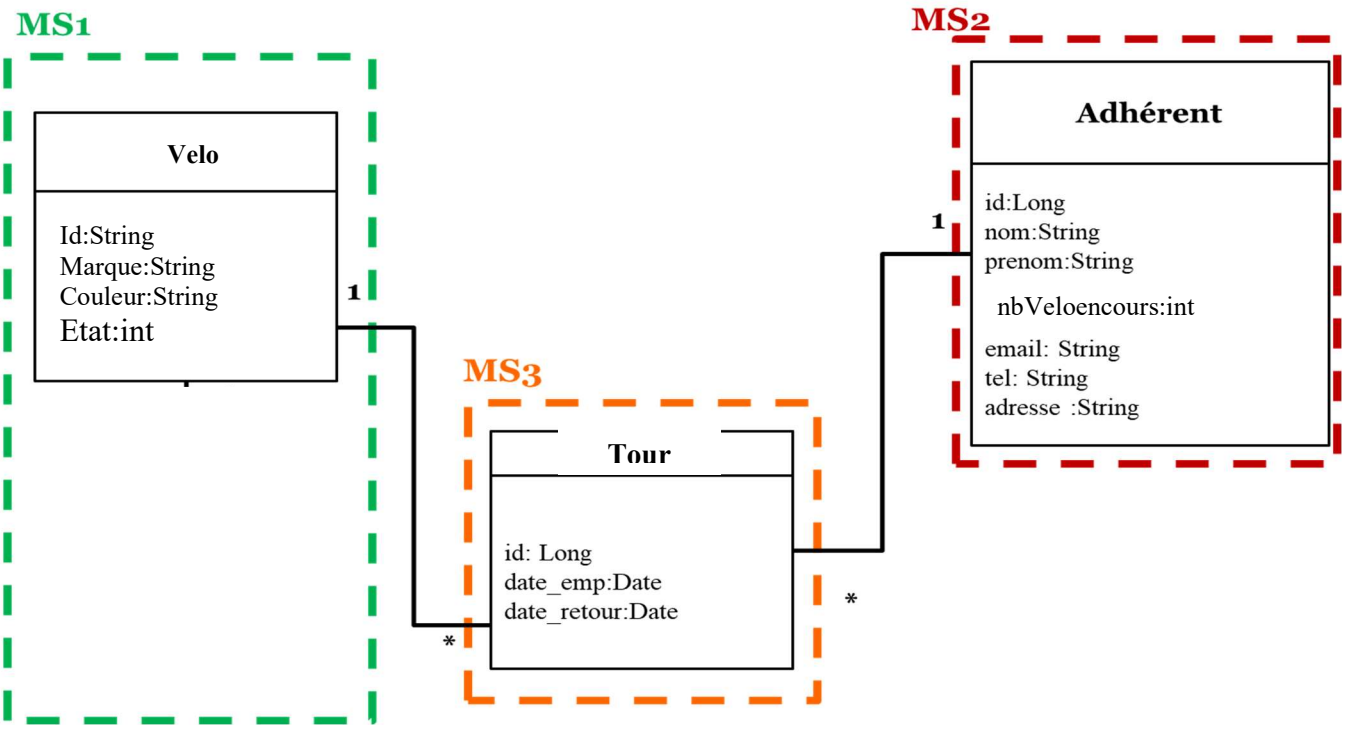
Etude de Cas

Nous cherchons à développer une application Web permettant la gestion de location de vélos à une station vélo comme le montre le diagramme de classes ci-dessous.

Un vélo est caractérisé par son id, sa marque, sa couleur et son état (non emprunté 0, emprunté 1).

Un vélo peut être emprunté par plusieurs adhérents où chaque adhérent est caractérisé par son nom, prénom, cin, email, adresse.

Dans cette application, l'agent peut enregistrer de nouveaux adhérents dans le système, ces derniers ayant la possibilité de rechercher des vélos, effectuer des tours de vélos et retourner les vélos avant la date d'échéance du tour.



Nous avons distingué 2 acteurs : l'agent et l'adhérent. Cette application Web doit permettre à l'agent de:

- s'authentifier saisir son login et son mot de passe pour qu'il puisse accéder à son espace privé,
- gérer les vélos (ajout, mise à jour, consultation),
- gérer les adhérents,
- gérer les tours et les retours de vélos

àl'adhérent de :

- s'authentifier,
- chercher des vélos
- faire des tours de vélos
- consulter la liste de ses tours
- retourner des vélos avant la date d'échéance.

Travail demandé

NB : Dans tout le travail, on suppose que le back end existe sur localhost :8080/api

1) a) Donner le code de la classe **Adherant.ts**

(selon le diagramme de classes ci-dessus y compris les attributs ajoutés par les relations entre classes)

```
export interface Adherant
{
id:number;
nom:String;
.....
tab_Tour:Tour []
}
```

b) Considérons le service **TourService.ts** . Développer le code des méthodes suivantes :

getToursByIDAdherant(IdAderant)...:Observable<Tour[]>

```
return this.http.get<Tour[]>('localhost:8080/api/Adherant/${idAdherant}')
```

getAllTours()...:Observable <Tour[]>

```
return this.http.get<Tour[]>('localhost:8080/ap/Tour')
```

2/ On se propose maintenant de créer un Adhérant à l'aide du composant **AdherantCreateComponent**. On vous donne ci-dessous le code de la page **AdherantCreateComponent.html**. En considérant le code donné ci dessous, donnez le code de **AdherantCreateComponent.ts** initialisant le formulaire et récupérant les données du formulaire de création **qui est affiché sur une boite de dialogue**, créant l'adhérant à l'aide de la méthode `ajouterAdherent(Adherant a)` **du service** et redirigeant vers la page **/Adherants**. **[Donnez aussi le code de la méthode** `ajouterAdherent (Adherant a)` **du service AdherentService]**

```
<form [formGroup]="form" >

<mat-form-field>

<input matInput placeholder="nom" formControlName="nom">

</mat-form-field>

<mat-form-field>

<input matInput placeholder="prenom" formControlName="prenom" >

// le reste des  attributs

<mat-dialog-actions>

<button (click)=ajouterAdherant()> </button>          </mat-dialog-actions>

</form>

export class AdherantCreateComponent

{   form !:FormGroup

Constructor( private DialogRef:MatDialogRef<AdherantCreateComponent>,private
             AS:AdherantService ,private router:Router
{

}
 //initialiser le form
 ngOnInit(this.form=new FormGroup({nom:new FormControl(null),
 )}}

ajouterAdherant()
{
this.AS.AjouterAdherant(this.form.value).subscribe(()=>{
this.router.navigate(['/Adherant'])
})
}

}
```

```
Export class Adherant Service
{constructor(private http:HttpClient){}
ajouterAdherent (Adherant a) Observable<void>
{return.this.http.post<void>('  /Adherant',a)
}}
```

3/a) On se focalise maintenant **sur la partie Adherent,** ce dernier, une fois authentifié, il visualise la liste des vélos **disponibles** dans l'agence. Donner le code **ts** du composant **VéloListComponent** permettant de **récupérer la liste des vélos disponibles** et **recherchant un vélo à travers son ID** tout en se servant des méthodes `chercherVéloparId(Long VeloId)` et `chercherTousLesVelos()` **du serviceVelo.**

```
export class VeloListComponent
{tab_VeloDispo:Velo[]=[]  VeloCherche:Velo}
Constructor( private VS/VeloService
{
}
GetAllVelo()
{
this.VS.chercherTousLesVelos()subscribe((response)=>
{this_VeloDispo=response:filter(e=>e:etat==0)}
chercherVeloById(idVelo:number){
this.VS.chercherVeloparId(idVelo).subscribe((V)=>this.VeloCherche=V)
}}
Export class VeloService
{ chercherTousLesVelos():Observable<Velo[]>
  {
  return this.http.get<Velo[]>('  /Velo');
  }
  chercherVeloById(idVelo: number) :Observable<Velo>{
   return
  this.http.get<Velo>('  /Velo/${idVelo}');
  }
```

.....

.....}

b) Dans la liste des vélos, le click sur le bouton **faire un tour** dans un des vélos qui se trouve sur la liste permet d'appeler la méthode **`Tour_Velo(Long Veloid, Long adherentid)`**.

Donner le code de cette méthode dans le composant VeloListComponent.

Cette méthode appelle une méthode dans le service **`TourService`** et redirige l'adhérant vers la page des tours là ou il peut visualiser ses tours y compris le nouveau tour.

Export Class VeloListComponent

{.....

`Constructor(` private TS:TourService,private router:Router

{.....

.....

.....}

`Tour_Velo (Long Veloid, Long adherentid)`

{this.TS.affect(VeloId,adherantid).subscribe(()=>)this.router.navigate(['/tour'])

affect (VeloId:number,adherantId::number):Observable<void>
return this.http.post<void>('/Tour/${VeloId}/${adherantId})

.....

return this.http.patch
('...../${idVelo},{etat=1})

.....

.....

.....

.....

.....

.....

.....

.....

.....}}

`Export class TourService`

{.....

.....

.....

.....

.....}

4/ On se propose de développer la partie **Dashboard** de cette application, On vous demande de donner le code TS permettant de générer une **courbe à secteurs** permettant d'afficher le pourcentage **des vélos en tour par rapport au nombre total de vélos**. Donnez aussi le code TS qui permet d'afficher dans une **courbe en battons le nombre d'emprunts par adhérent.**

```
<canvas baseChart
[type]="'pie'"
[dataSets]="chartData1"
[Labels]="chartLabels1>
</canvas>
```

```
<canvas baseChart
[type]="bar"
[dataSets]="chartData2">
[Labels]="chartLabels2
</canvas>
```

```
Export class Dashboard
Constructor(  private VS:VeloService  )
{
this.VS.chercherTousLesVelos().subscribe((response)=>
{this.VTour=response.filter(e=>.....}
.....
.....
.....
chartData1:ChartDataSet[]=[{
Data:.....
.....}]
chartLabels1:string[]=['VTour','VNonTour'];.....
.....
chartData2:ChartDataSet[]=[{
Data:.....
.....
.....}]
chartLabels2:string[]
.....
}
```